In [5]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Data collection and preprocessing

In [6]:
```
data = pd.read_csv(r"sonar data.csv", header=None)
data.head()
```

Out[6]:

	0	1	2	3	4	5	6	7	8	9	...	51
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0027
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0084
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0232
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0121
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0031

5 rows × 61 columns

In [7]:
```
# number of rows and columns
data.shape
```

Out[7]:
```
(208, 61)
```

In [8]:
```
data.describe()
```

Out[8]:

	0	1	2	3	4	5	6
count	208.000000	208.000000	208.000000	208.000000	208.000000	208.000000	208.000000
mean	0.029164	0.038437	0.043832	0.053892	0.075202	0.104570	0.121747
std	0.022991	0.032960	0.038428	0.046528	0.055552	0.059105	0.061788
min	0.001500	0.000600	0.001500	0.005800	0.006700	0.010200	0.003300
25%	0.013350	0.016450	0.018950	0.024375	0.038050	0.067025	0.080900
50%	0.022800	0.030800	0.034300	0.044050	0.062500	0.092150	0.106950
75%	0.035550	0.047950	0.057950	0.064500	0.100275	0.134125	0.154000
max	0.137100	0.233900	0.305900	0.426400	0.401000	0.382300	0.372900

8 rows × 60 columns

In [10]:
```
data[60].value_counts()
```

Out[10]:
```
60
M    111
R     97
Name: count, dtype: int64
```

In [11]:
```
# M - Mine
# R - Rock
```

In [12]:
```
data.groupby(60).mean()
```

Out[12]:

60	0	1	2	3	4	5	6	7
M	0.034989	0.045544	0.050720	0.064768	0.086715	0.111864	0.128359	0.149832	0.21349
R	0.022498	0.030303	0.035951	0.041447	0.062028	0.096224	0.114180	0.117596	0.13739

2 rows × 60 columns

In [13]:
```
# separating data and labels
X = data.drop(60, axis=1)
y = data[60]
```

In [17]:
```
X.head()
```

Out[17]:

	0	1	2	3	4	5	6	7	8	9	...	50
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0232
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0125
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0033
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0241
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0156

5 rows × 60 columns

In [15]:
```
y.head()
```

Out[15]:
```
0    R
1    R
2    R
3    R
4    R
Name: 60, dtype: str
```

Training and test data

In [18]:
```
X_train,X_test,y_train,y_test = train_test_split(X,y, test_size=0.2, random_state=4
```

Model Training -- LogisticRegression

In [19]:
```
model = LogisticRegression()
```

In [20]:
```
# train the model with training data
model.fit(X_train,y_train)
```

Out[20]:

LogisticRegression

▶ Parameters

▶ Fitted attributes

Model Evaluation

In [21]:
```
# check accuracy on training data
X_train_pred = model.predict(X_train)
```

```
training_data_acc = accuracy_score(y_train, X_train_pred)
print("Accuracy on training data:", training_data_acc)
```

```
Accuracy on training data: 0.8373493975903614
```

In [22]:
```
# check accuracy on Test data
X_test_pred = model.predict(X_test)
test_data_acc = accuracy_score(y_test, X_test_pred)
print("Accuracy on Test data: ", test_data_acc)
```

```
Accuracy on Test data:  0.7857142857142857
```

Making a predictive System

In [28]:
```
input_data = (0.0200,0.0371,0.0428,0.0207,0.0954,0.0986,0.1539,0.1601,0.3109,0.2111
# target is 'R' - Rock
# chaning the input_data into numpy array
input_data_as_np_array = np.array(input_data)

# reshape the numpy array as we are predicting for one instance
input_data_reshaped = input_data_as_np_array.reshape(1,-1)

prediction = model.predict(input_data_reshaped)
print(prediction)

if prediction[0] == 'R':
    print("The object is Rock")
else:
    print("The object is Mine")
```

```
['R']
The object is Rock
```

In [29]:
```
input_data = (0.0056,0.0267,0.0221,0.0561,0.0936,0.1146,0.0706,0.0996,0.1673,0.1859
# target is 'M' - Mine
# chaning the input_data into numpy array
input_data_as_np_array = np.array(input_data)

# reshape the numpy array as we are predicting for one instance
input_data_reshaped = input_data_as_np_array.reshape(1,-1)

prediction = model.predict(input_data_reshaped)
print(prediction)

if prediction[0] == 'R':
    print("The object is Rock")
else:
    print("The object is Mine")
```

```
['M']
The object is Mine
```

In [ ]: